CG3207 Assignment 3 — Study Notes

Multiplication and division in the MCycle unit, radix-2 Booth, and putting it all into the RISC-V CPU

These notes go from the basics up. Every worked example was traced with a script that copies the hardware step by step, so the bit patterns are exactly what the circuit produces.

Contents

§	Section
0	The big picture: which files changed
1	Background: two's complement and sign extension
2	How the MCycle unit works (the template's machinery)
3	The original multiplier (shift-and-add)
4	Task 1: restoring division, unsigned then signed
5	Task 3: radix-2 Booth multiplication (and the Task 4 visualisation)
6	Task 2: putting MCycle into the CPU
7	The demo program
8	How everything was tested
9	Questions you're likely to be asked

0. The big picture

File	Task	What changed
`MCycle.v`	1, 3	Added division (Task 1); radix-2 Booth for signed multiply (Task 3, Hin Lok's design)
`test_MCycle.v`	1	Turned into a self-checking testbench; added a post-synthesis version
`Decoder.v`	2	Recognises the M instructions and generates MCycle's control signals
`RV.v`	2	Instantiates MCycle, stalls the CPU while it's busy, selects its result
`Wrapper.v`	2	Instruction and data memories enlarged from 128 to 256 words
`test.asm`, `AA_IROM.mem`, `AA_DMEM.mem`	2	New demo program
`test_Wrapper.v`	2	New self-checking testbench for the program
`cpu_check.py`	2	Updated so the Assignment 2 regression test knows about MCycle
`booth_visualisation.html`	4	Interactive step-by-step visualisation of the Booth multiplier
`docs/CG3207_Task1_Division_Dataflow.html`	1	Study aid: one division traced signal by signal through MCycle.v (Qi Ye)

1. Background: two's complement and sign extension

Every signed number in the CPU is in **two's complement**. In a w-bit number:

- The top bit has weight $\mathbf{-2^{w-1}}$.
- Every other bit i has its normal weight $\mathbf{+2^{i}}$.

For 4 bits:

Bits	Unsigned	Signed
0111	7	7 (largest positive)
1000	8	−8 (most negative)
1101	13	−8 + 4 + 1 = −3
1111	15	−1

Three facts are used everywhere in this assignment.

Fact 1: negation is "invert all bits, then add 1". −(0011) = 1100 + 1 = 1101 = −3. The hardware can therefore subtract X by adding the inverted X with a carry-in of 1. That's how synthesis builds the Booth multiplier's A − M.

Examples on 32 bits:

Value	Step	Hex
1		`00000001`
	invert	`FFFFFFFE`
−1	+1	**`FFFFFFFF`**
15		`0000000F`
	invert	`FFFFFFF0`
−15	+1	**`FFFFFFF1`**

To read a negative number, negate it back. `FFFFFFEE` → invert → `00000011` → +1 → `00000012` = 18, so `FFFFFFEE` = −18. A quick shortcut for small negatives: `FFFFFFFF – FFFFFFEE = 0x11 = 17`, so it is −1 − 17 = −18.

Fact 2: sign extension preserves the value. To widen a signed number, copy its top bit into the new positions: 1101 (−3) becomes 11111101, which is still −3 in 8 bits. To widen an unsigned number, fill with zeros: 1101 (13) becomes 00001101. Both multipliers do this, and MCycleOp[0] picks which (0 = signed, so copy the sign; 1 = unsigned, so fill with 0).

Fact 3: the most negative number has no positive partner. In 4 bits the range is −8 … +7, so −(−8) doesn't fit as a signed number: invert 1000 to get 0111, add 1, and you get 1000 again. It does fit as an unsigned number (1000 = 8). This matters for division (Section 4.7).

Arithmetic shift right (written `>>>`) shifts right and copies the sign bit into the top, so it divides a signed number by 2 rounding down: 1100 (−4) >>> 1 = 1110 (−2). The Booth multiplier shifts its {A, Q, Q_{-1}} register this way.

Signed vs unsigned meaning of the same bits. The same bits mean different numbers depending on whether an instruction treats them as signed or unsigned:

Bits	Signed (`div`, `mul`, `mulh`)	Unsigned (`divu`, `mulhu`)
`FFFFFFFF`	−1	4,294,967,295
`FFFFFFFA`	−6	4,294,967,290
`80000000`	−2,147,483,648	2,147,483,648

Example: −6 ÷ 3 with `div` gives −2 (`FFFFFFFE`). With `divu`, the bits `FFFFFFFA` are read as 4,294,967,290, and 4,294,967,290 ÷ 3 = 1,431,655,763 remainder 1 = **`55555553`**. Check: `0x55555553 × 3 = 0xFFFFFFF9`, + 1 = `0xFFFFFFFA` ✓. (It looks like a pattern because `FFFFFFFF ÷ 3 = 55555555` exactly, since `5 × 3 = F` in every hex digit; −5 with `divu` gives the same quotient with remainder 2.)

---

2. How the MCycle unit works (the template's machinery)

Before the algorithms, you need to know how MCycle runs any operation over several cycles. This machinery was kept from the template, so it's worth understanding properly.

2.1 The ports

```
input  Start          // "please start an operation" (from the Decoder)
input  [1:0] MCycleOp // 00 signed mul, 01 unsigned mul, 10 signed div, 11 unsigned div
input  Operand1, Operand2
output Result1        // low word of product / quotient
output Result2        // high word of product / remainder
output Busy           // 1 while working
```

2.2 The state machine (`IDLE_PROCESS`)

There are only two states: **IDLE** and **COMPUTING**.

```
IDLE      --Start=1-->  COMPUTING   (Busy = 1 immediately)
COMPUTING --done=0-->   COMPUTING   (Busy = 1)
COMPUTING --done=1-->   IDLE        (Busy = 0)
```

`Busy` is computed **combinationally**, from the current state and the inputs. In IDLE, Busy goes to 1 *in the same cycle* that Start rises, without waiting for a clock edge. A machine whose output depends on its inputs like this is a **Mealy machine**. It's essential for the CPU: the instant a `mul` instruction appears, Busy goes high and freezes the PC before the next clock edge can advance it.

2.3 The "next-value" coding style (`COMPUTING_PROCESS`)

Every register `x` has a partner `n_x`, meaning "what x will become at the next clock edge". The combinational block:

1. starts with `n_x = x` for every register (hold by default);
2. modifies the `n_` values step by step, like a little program;
3. the clocked block copies every `n_x` into `x` at the clock edge, in one place.

```
always @(posedge CLK) begin
    state <= n_state ; count <= n_count ; temp_sum <= n_temp_sum ; ...
end
```

The benefit is that several steps happen in one cycle, in order, inside one combinational block. For example, the start cycle first *loads* the operands into `n_` variables, and the very next lines *already*

run the first iteration on those loaded values. The hardware is a chain of logic: load-mux → adder → shift → register input.

2.4 The start condition

```
if( RESET | (n_state == COMPUTING & state == IDLE) ) begin
    n_count = 0 ; n_temp_sum = 0 ;
    n_shifted_op1 = ... Operand1 ... ;  // capture the operands
    ...
end
```

`n_state == COMPUTING & state == IDLE` means "we are in IDLE and about to start". This is the only moment Operand1 and Operand2 are read. After that, MCycle works only on its own registered copies. That's important in the CPU (Section 6.6).

2.5 The guard

```
if( n_state == COMPUTING ) begin  ... one iteration ...  end
```

An iteration only runs if we will still be computing after this edge. In the cycle where Busy falls (state = COMPUTING, done = 1, so n_state = IDLE), no iteration runs and the results are held. That's the cycle in which the CPU writes the result to the register file, so the results must not change during it.

2.6 `count` and `done`

`count` counts iterations. When the last iteration runs, the code sets `n_done = 1`. `done` is registered, so it becomes 1 at the next edge, and then Busy falls.

2.7 Cycle-by-cycle timeline (32-bit Booth multiply, 32 iterations)

Cycle	state	Busy	What happens
1	IDLE	1 (because Start = 1)	Operands loaded **and** iteration 0 computed; stored at the edge
2	COMPUTING	1	iteration 1
…	COMPUTING	1	…
32	COMPUTING	1	iteration 31 (the last), `n_done = 1`, final results stored
33	COMPUTING, done = 1	**0**	no iteration; results stable; the CPU writes rd and the PC moves on
34	IDLE	depends on the next instruction	ready again

So the latency is **iterations + 1**: 32 + 1 = 33 for Booth multiply and for division, and 64 + 1 = 65 for the original signed multiply. That +1 is why the board measures 33, not 32.

---

3. The original multiplier (shift-and-add)

You need this to see what the enhancement improves.

3.1 Pencil-and-paper binary multiplication

Multiplication is repeated addition of shifted copies. In binary each multiplier bit is 0 or 1, so each step either adds a shifted copy of the multiplicand or adds nothing.

Example, unsigned 13 × 6 (4-bit: 1101 × 0110):

```
    1101      (13, multiplicand)
  x 0110      (6, multiplier)
  ------
    0000      bit 0 = 0 -> add nothing
   1101       bit 1 = 1 -> add 13 shifted left 1
  1101        bit 2 = 1 -> add 13 shifted left 2
 0000         bit 3 = 0 -> add nothing
--------
01001110      = 78 = 0x4E
```

3.2 How the template's hardware does it

- `shifted_op1` holds the multiplicand and shifts **left** 1 each cycle.
- `shifted_op2` holds the multiplier and shifts **right** 1 each cycle, so the bit currently in use is always bit 0.
- `temp_sum` accumulates the result.

```
if( n_shifted_op2[0] ) n_temp_sum = n_temp_sum + n_shifted_op1 ;  // add if bit = 1
n_shifted_op2 = {1'b0, n_shifted_op2[2*width-1:1]} ;              // next bit to bit 0
n_shifted_op1 = {n_shifted_op1[2*width-2:0], 1'b0} ;              // multiplicand x 2
```

One multiplier bit per cycle, so **w cycles** for unsigned.

3.3 How it handles signed numbers, and why that costs 2w cycles

The template sign-extends both operands to 2w bits and multiplies them as if they were unsigned 2w-bit numbers, keeping only the low 2w bits.

This works because the low 2w bits of a product only depend on the low 2w bits of the inputs (arithmetic is "modulo 2^{2w}"), and a sign-extended number is congruent to its signed value modulo 2^{2w}.

Example, −3 × 6:

```
11111101 (-3) x 00000110 (6)
= 11111101<<1 + 11111101<<2 = 11111010 + 11110100 = 1 11101110
low 8 bits = 11101110 = 0xEE = -18
```

The cost: the multiplier now has **2w bits**. For a negative multiplier, the extra w bits are all 1s and each needs an addition, so the loop must run **2w = 64 cycles** for 32-bit signed. It also needs a **2w-bit adder** (64-bit) and two 2w-bit shift registers. That's the weak spot Task 3 attacks.

---

4. Task 1: restoring division

4.1 Long division, decimal first

Dividing 13 by 3 by hand: bring down one digit at a time, ask "how many times does 3 go into what I have?", write that digit of the quotient, subtract, and carry the remainder forward.

4.2 In binary the question is only "yes or no"

In binary each quotient digit is 0 or 1, so each step is one question: **is the current remainder ≥ the divisor?**

- If yes, the quotient bit is 1 and you subtract.
- If no, the quotient bit is 0 and you leave the remainder alone.

4.3 The register layout

MCycle puts the remainder **R** and the dividend/quotient **Q** side by side in one 2w-bit register (`temp_sum`):

```
temp_sum = { R (upper w bits) , Q (lower w bits) }
```

- At the start: R = 0, Q = dividend, and D (the divisor) is kept in `shifted_op2`.
- Each cycle:
  1. **Shift {R, Q} left by 1.** The top bit of Q (the next dividend bit, "brought down") moves into R. A free slot appears at the bottom of Q.
  2. **Trial subtract:** compute R − D.
  3. If R − D ≥ 0, keep the difference as the new R and put **1** in the free slot.
  4. Otherwise, keep R unchanged ("restore") and put **0** in the free slot.

After w cycles, all the dividend bits have moved into R and been replaced by quotient bits. Q now holds the **quotient** and R holds the **remainder**.

It's called *restoring* division because when the trial subtraction fails, the original R is kept, i.e. restored.

4.4 Worked example: 13 ÷ 3 in 4 bits

Start: R = 0000, Q = 1101 (13), D = 0011 (3).

Cycle	After shift: R (5 bits)	Q	R − D	Decision	New R	New Q
0	00001	101_	1 − 3 = −2	restore, q = 0	0001	1010
1	00011	010_	3 − 3 = 0	keep, q = 1	0000	0101
2	00000	101_	0 − 3 = −3	restore, q = 0	0000	1010
3	00001	010_	1 − 3 = −2	restore, q = 0	0001	0100

Result: Q = 0100 = **4**, R = 0001 = **1**. Check: 4 × 3 + 1 = 13 ✓.

Follow the quotient bits as they enter at the right: 0, 1, 0, 0, giving 0100.

4.5 Why R needs one extra bit

After the shift, R = 2 × (old R) + (new bit). Old R is always less than D, so the new R is at most 2D − 1. If D is large (say D = 1111 = 15), the shifted R can be up to 29, which needs **5 bits** in a 4-bit design. If the top bit were thrown away, the comparison would be wrong.

So the shift keeps the bit that falls out:

```
div_shifted = { n_temp_sum, 1'b0 } ;   // 2w+1 bits: the extra top bit is kept
```

The trial subtraction is done in **w + 2 bits**, so the top bit is a reliable sign (negative means R < D):

```
div_diff = { 1'b0, div_shifted[2*width : width] } - { 2'b00, n_shifted_op2[width-1:0] } ;
if( ~div_diff[width+1] )   // not negative -> R >= D
    n_temp_sum = { div_diff[width-1:0], div_shifted[width-1:1], 1'b1 } ;   // keep, q = 1
else
    n_temp_sum = { div_shifted[2*width-1:1], 1'b0 } ;                       // restore, q = 0
```

When R ≥ D, the difference is always less than D, so it fits back into w bits. When R < D, R itself is less than D and also fits. So R only needs w bits *between* cycles; the extra bit is only needed during the comparison.

This is 1 quotient bit per cycle, so **w = 32 iterations + 1 = 33 cycles**. Unsigned division (`divu`) is exactly this, with no further steps.

4.6 Signed division: magnitudes, then fix the signs

Restoring division only works on non-negative numbers. For signed division:

1. **At the start:** take the magnitudes, |dividend| and |divisor|, and remember two signs:
   - **Quotient sign:** negative if the operand signs differ (− ÷ + or + ÷ −).
   - **Remainder sign:** the same as the **dividend's** sign.
2. **Divide** the magnitudes with the unsigned algorithm above.
3. **At the end:** negate the quotient and/or the remainder as remembered.

```
op1_neg = ~MCycleOp[0] & Operand1[width-1] ;      // only signed division looks at signs
op2_neg = ~MCycleOp[0] & Operand2[width-1] ;
dividend_mag = op1_neg ? -Operand1 : Operand1 ;
divisor_mag  = op2_neg ? -Operand2 : Operand2 ;
...
n_neg_quotient  = (op1_neg ^ op2_neg) & (|Operand2) ;
n_neg_remainder = op1_neg ;
...
n_Result1 = n_neg_quotient  ? -quotient  : quotient ;
n_Result2 = n_neg_remainder ? -remainder : remainder ;
```

Why these sign rules? RISC-V (like C) defines signed division as **rounding towards zero**, and it requires dividend = quotient × divisor + remainder.

a ÷ b	magnitudes	quotient	remainder	check
7 ÷ 3	7 ÷ 3 = 2 r 1	2	1	2×3+1 = 7
−7 ÷ 3	2 r 1	**−2** (signs differ)	**−1** (dividend negative)	−2×3−1 = −7
7 ÷ −3	2 r 1	**−2**	**1** (dividend positive)	−2×−3+1 = 7
−7 ÷ −3	2 r 1	**2** (signs equal)	**−1**	2×−3−1 = −7

These are your test cases 0–3, and the board shows exactly these values.

4.7 The edge cases

$-2^{w-1} \div -1$ (overflow). The true answer $+2^{31}$ doesn't fit in 32-bit signed. RISC-V says the result must be -2^{31} with remainder 0. Here's what the hardware does with 4 bits (−8 ÷ −1):

- |−8| = −(1000) = 1000. As an unsigned magnitude that's 8 (Fact 3), so it's fine.
- 8 ÷ 1 = 8 = 1000, remainder 0.
- The signs are equal, so the quotient isn't negated. Result: 1000, which read as signed is −8.

So the required RISC-V answer comes out naturally, with no special-case hardware. That's test case 4 (`80000000`).

Division by zero. The manual says you can assume it never happens. Still, it's worth knowing what the hardware does:

- With D = 0, every trial subtraction R − 0 ≥ 0 succeeds, so every quotient bit is 1. The quotient is all ones, and R ends as the dividend.
- RISC-V specifies exactly that: quotient = all ones (−1), remainder = dividend.
- For signed division, the `& (|Operand2)` term stops the quotient from being negated when the divisor is 0, so −5 ÷ 0 also gives −1, not +1. `|Operand2` is OR of all bits, which is 1 if the divisor is non-zero.

That's why case 9 (5 ÷ 0) shows `FFFFFFFF` and remainder 5, matching RARS.

---

5. Task 3: radix-2 Booth multiplication

The group's Task 3 enhancement is Hin Lok's radix-2 Booth multiplier for **signed** multiplication. (An earlier radix-4 version is still in the git history, commit `6c27ed1`, if you want to compare.)

5.1 The goal

The original multiplier can only add *positive* shifted copies of the multiplicand. To handle a negative multiplier it sign-extends both operands to 2w bits and runs **2w = 64 iterations** with a **64-bit adder** (Section 3.3). Booth's algorithm reads the multiplier as a signed number directly, so it needs only **w = 32 iterations**: 65 → 33 cycles for a 32-bit signed `mul`.

5.2 The idea: runs of 1s

A run of 1s in the multiplier can be replaced by one subtraction and one addition:

```
0011 1100  =  0100 0000 - 0000 0100      (60 = 64 - 4)
```

Instead of adding M four times (once per 1), Booth subtracts M at the **start** of the run (the lowest 1) and adds M just **after its end**. Inside a run, and inside a run of 0s, nothing happens.

The hardware spots the start and end of runs by looking at **two adjacent bits** of the multiplier: the current bit Q_0 and the bit to its right, Q_{-1} (which starts as an imaginary 0).

Q_0 Q_{-1}	Meaning	A becomes
0 0	inside a run of 0s	A (unchanged)
0 1	just after a run of 1s ended	A + M
1 0	a run of 1s starts here	A − M
1 1	inside a run of 1s	A (unchanged)

5.3 Why it works (and why it handles signed numbers)

The rule says the digit used at bit position i is $d_i = Q_{i-1} - Q_i$ (with $Q_{-1} = 0$): 01 → +1, 10 → −1, 00 and 11 → 0. Add them up with their weights:

$$\Sigma\, d_i \cdot 2^i = \Sigma\, (Q_{i-1} - Q_i) \cdot 2^i$$

Each bit Q_i appears twice: once as $-Q_i \cdot 2^i$ (in digit i), and once as $+Q_i \cdot 2^{i+1}$ (in digit i+1, where it is the "previous" bit). Together that's $+Q_i \cdot 2^i$, its normal weight ✓.

The **top bit** Q_{w-1} has no digit above it, so it keeps only the $-Q_{w-1} \cdot 2^{w-1}$ part. That is **exactly the two's complement sign weight** (Section 1) ✓. So Booth recoding is an exact rewrite of a signed number, and multiplying by it gives the signed product with no sign extension to 2w bits.

Check with Q = 5 = 0101: pairs (Q_i, Q_{i-1}) from the right are (1,0), (0,1), (1,0), (0,1), giving digits −1, +1, −1, +1. Value: −1 + 2 − 4 + 8 = 5 ✓.

5.4 The hardware

Four registers (all in MCycle.v, prefixed `booth_`):

Register	Bits (w = 32)	Starts as	Role
M	w+1 = 33	multiplicand, sign-extended	the value added or subtracted
A	w+1 = 33	0	accumulator: the upper half of the product
Q	w = 32	multiplier	the bits still to be read; the lower half of the product
Q_{-1}	1	0	the last bit shifted out of Q

Each clock edge does, in one combinational pass:

1. **Decide:** look at $\{Q_0, Q_{-1}\}$.
2. **Add/subtract:** A = A + M (01), A − M (10), or unchanged (00/11). One (w+1)-bit adder/subtractor.
3. **Shift:** shift the whole $\{A, Q, Q_{-1}\}$ right by one bit, **copying A's sign bit** into the top (arithmetic shift). A's lowest bit moves into the top of Q, and Q's lowest bit becomes the new Q_{-1}.

```
case( {n_booth_Q[0], n_booth_Qminus1} )
    2'b01:   n_booth_A = n_booth_A + n_booth_M ;   // end of a run of 1s: add M
    2'b10:   n_booth_A = n_booth_A - n_booth_M ;   // start of a run of 1s: subtract M
    default: ;                                     // 00 / 11: A unchanged
endcase
booth_shift = { n_booth_A, n_booth_Q, n_booth_Qminus1 } ;
booth_shift = { booth_shift[2*width+1], booth_shift[2*width+1:1] } ;   // arithmetic shift right
by 1
n_booth_A = booth_shift[2*width+1:width+1] ;
n_booth_Q = booth_shift[width:1] ;
n_booth_Qminus1 = booth_shift[0] ;
```

After w edges, all the multiplier bits have been shifted out of Q and replaced by the low half of the product. The **product is {A[w−1:0], Q}**:

```
n_Result2 = n_booth_A[width-1:0] ;   // high word
n_Result1 = n_booth_Q ;              // low word
```

The shift plays the role of "shift the multiplicand left" in pencil-and-paper multiplication: instead of moving M left each step, the partial product moves right, so the adder only ever works on the top part (w+1 bits instead of 2w).

5.5 Why A and M have one extra bit

−M must be representable. If M is the most negative number (−8 in 4 bits, 1000), then −M = +8 does not fit in 4 signed bits (Fact 3). With 5 bits, M = 11000 and −M = 01000 = +8 fits. The extra bit also

keeps the sign of A correct after an addition or subtraction overflows 4 bits. It is dropped at the end (the product is {A[w−1:0], Q}).

5.6 Worked example: −3 × 5 in 4 bits

M = −3 = **11101** (5 bits), Q = 5 = **0101**, A = 00000, Q_{-1} = 0. −M = +3 = 00011. Four edges.

Edge	$\{Q_0, Q_{-1}\}$	Operation	A before shift	A after	Q after	Q_{-1} after
start	—	load	—	00000	0101	0
1	1 0	A − M = 0 + 3	00011	00001	1010	1
2	0 1	A + M = 1 − 3	11110	11111	0101	0
3	1 0	A − M = −1 + 3	00010	00001	0010	1
4	0 1	A + M = 1 − 3	11110	11111	0001	0

How edge 1's shift works: $\{A, Q, Q_{-1}\}$ = 00011 0101 0 → shift right, copy sign 0 → 00001 1010 1. A's low bit (1) went into Q's top; Q's low bit (1) became Q_{-1}.

Product = {A[3:0], Q} = **1111 0001 = 0xF1 = −15** ✓.

The hard case, −8 × −1: M = 11000, −M = 01000 (+8, only possible because of the extra bit). Q = 1111. Edge 1 sees 10 → A = 0 + 8 = 01000 → shift → 00100. Edges 2–4 see 11 → no add, just shifts: 00010, 00001, 00000, while Q fills with 1000. Product = 0000 1000 = **+8** ✓.

5.7 What improved, and what it costs

32-bit	Original	Radix-2 Booth	Why
Signed multiply	65 cycles	**33**	reads the multiplier as signed: 32 iterations instead of 64
Unsigned multiply	33 cycles	33	unchanged (original shift-and-add)
Adder used for signed multiply	64-bit	**33-bit** add/ subtract	only A (the top part) is added to
Whole MCycle (Yosys estimate)	905 LUTs / 268 FF	1016 LUTs / 367 FF	Booth's A, M, Q registers and adder are added next to the template's ones, which unsigned multiply still uses

So it is a **speed-for-area trade-off**: signed multiplication is about 2× faster for roughly 12% more LUTs and 37% more flip-flops. The manual explicitly allows "the trade-off between the two". (Replace the Yosys estimates with Vivado's utilisation numbers for the slides.)

`BOOTH = 0` brings the original signed multiplier back, so the testbench and the post-synthesis simulation compare both versions from one file.

5.8 The Task 4 visualisation (`booth_visualisation.html`)

Hin Lok's single-file page lets you enter two operands (decimal, or a hex pattern copied from the board), pick 4, 8 or 32 bits, and step one clock edge at a time. It shows M, A, Q and Q_{-1} bit by bit (changed bits in yellow), the decision table row in use, a datapath diagram highlighting the path used at that edge (select +M / −M / 0 → adder → arithmetic shift → registers), the full trace, the cycle count (Booth w+1 vs template 2w+1), and State/Busy. Its model was checked against the RTL: the register values after every edge are identical to a Verilog simulation of MCycle.v.

6. Task 2: putting MCycle into the CPU

6.1 How M instructions are encoded

M instructions use the same opcode as `add` / `sub`, **0110011 (R-type)**. They're told apart by **funct7 = 0000001**, which is 0000000 for add and 0100000 for sub. funct3 then picks the operation:

funct3	Instruction	Meaning	MCycleOp	Which result
000	mul	low 32 bits of a × b	00	Result1
001	mulh	high 32 bits, signed × signed	00	Result2
010	mulhsu	high 32 bits, signed × unsigned	(not supported, NOP)	—
011	mulhu	high 32 bits, unsigned × unsigned	01	Result2
100	div	signed quotient	10	Result1
101	divu	unsigned quotient	11	Result1
110	rem	signed remainder	10	Result2
111	remu	unsigned remainder	11	Result2

Why `mul` doesn't care about signed or unsigned: the low 32 bits of a product are the same either way, because of the modulo argument in 3.3. Only the *high* word differs, which is why there are separate `mulh` and `mulhu`.

Why rem/mulh were free to add: MCycle always computes both words (product low and high, or quotient and remainder). Supporting them only needed one more mux select bit. `mulhsu` would need MCycle to treat the two operands differently, so it was left out; the manual says these extras are optional.

6.2 Decoder changes

The original Decoder looked only at `Funct7[5]` for R-type, so `mul` (funct7 = 0000001) would have decoded as `add`. Now:

```
OP_DPREG: begin
    if( Funct7 == FUNCT7_M ) begin       // 0000001: an M instruction
        case( Funct3 ) ... set MCycleOp and MResultSel per the table ... endcase
        if( Funct3 != 3'b010 ) begin     // everything except mulhsu
            RegWrite  = 1 ;
            MStart    = 1 ;              // start MCycle
            MCycleSel = 1 ;              // write MCycle's result, not the ALU's
        end
    end
    else begin                           // normal add/sub/and/...
        RegWrite = 1 ; ALUControl = {Funct3, Funct7[5]} ;
    end
end
```

The four new outputs:

- **`MStart`**: connects to MCycle's Start.
- **`MCycleOp`**: which of the four operations.
- **`MCycleSel`**: the register write data comes from MCycle.
- **`MResultSel`**: Result1 (low word or quotient) versus Result2 (high word or remainder).

All four default to 0, so no other instruction can start MCycle by accident. Unsupported or illegal encodings write nothing, the same principle as in the Assignment 2 decoder.

6.3 The datapath in RV.v

```
MCycle #(.width(32)) MCycle1( .CLK(CLK), .RESET(RESET), .Start(MStart), .MCycleOp(MCycleOp),
                              .Operand1(RD1), .Operand2(RD2),
                              .Result1(MCycleResult1), .Result2(MCycleResult2), .Busy(Busy) ) ;

assign MCycleResult = MResultSel ? MCycleResult2 : MCycleResult1 ;
assign ExecResult   = MCycleSel  ? MCycleResult  : ALUResult ;   // the new multiplexer
assign Result       = MemtoReg   ? ReadData      : ExecResult ;
```

- The operands come straight from the register file: RD1 = rs1, RD2 = rs2, just like the ALU's register operands.
- `width` is set to 32 here. The default of 4 is only for unit testing.
- The new multiplexer sits between the ALU and the existing memory mux. The manual asked for "a multiplexer and a control signal to combine the outputs from the ALU and MCycle", and this is it.

6.4 The stall: `WE_PC = ~Busy`

In a single-cycle CPU, every instruction must normally finish in one clock. A `mul` takes 33, so the CPU has to **freeze** until MCycle is done.

```
assign WE_PC = ~Busy ;
```

- While Busy = 1, the program counter doesn't update.
- Because the PC doesn't change, instruction memory keeps outputting the **same instruction**.
- So the Decoder keeps generating the same control signals, and Start stays at 1.
- Everything downstream sees the `mul` instruction, cycle after cycle, until Busy falls.

This is why Busy must be combinational (Section 2.2). In the very first cycle of the `mul`, Busy must already be 1 before the clock edge, or the PC would move on to the next instruction immediately.

6.5 Holding the register write: `WE = RegWrite & ~Busy`

While the CPU is stalled, the Decoder still says RegWrite = 1, because the instruction is still `mul`. Without the gating, the register file would be written every stall cycle with whatever partial value MCycle currently outputs. So:

```
assign WE = RegWrite & ~Busy ;
```

The register is written **only once**, in the cycle where Busy falls. In that cycle:

- MCycle's results are already final (Section 2.5),
- the register file writes rd at the clock edge,
- and the PC updates to the next instruction, at the same edge.

That's exactly what a single-cycle instruction does, just later.

Is the gating strictly necessary? Here, functionally, not quite: MCycle captured its operands at the start, and the last write would be correct anyway. It's still the right design: no intermediate garbage ever appears in a register, and it's the behaviour you'll need when you pipeline the CPU in Assignment 4.

6.6 Why capturing the operands at the start matters

Take `div a6, a6, a1`, where rd is the same register as rs1. If writes weren't held off, *and* MCycle re-read Operand1 every cycle, a partial result written into a6 would change the dividend mid-calculation

and the answer would be wrong. MCycle reads its operands only in the start cycle (Section 2.4), and the register isn't written until the end, so this works. The demo program tests exactly this case (rd = rs1 for `div`, rd = rs2 for `mul`), and the testbench additionally changes the operand inputs randomly while MCycle is busy.

6.7 Timeline of `mul t0, a0, a1` followed by `mulh t1, a0, a1`

Cycle	PC points to	MCycle state	Busy	WE_PC	Register write
1	mul	IDLE → starting	1	0	no
2–32	mul	COMPUTING	1	0	no
33	mul	COMPUTING, done	0	1	**t0 written**
34	mulh	IDLE, Start = 1	1	0	no (a new operation starts)
…	mulh	…	…	…	…
66	mulh	done	0	1	**t1 written**

In cycle 33, Start is still 1 (it's still the `mul`), but the state machine goes COMPUTING + done → IDLE regardless of Start, so it doesn't restart by mistake. In cycle 34, the new instruction is a `mulh` and Start is 1 again, so a fresh operation begins. Back-to-back M instructions therefore work with no gap, which the program's self-check loop does seven times in a row.

6.8 Wrapper.v

The new program is 128 instructions and about 161 words of data (test inputs, expected results, display buffer). The default memories were 128 words each, so `IROM_DEPTH_BITS` and `DMEM_DEPTH_BITS` were raised from 9 to 10, giving 256 words each. Wrapper.v's own comments say to change these if your program is larger. Nothing else in the Wrapper changed.

---

7. The demo program (`test.asm`)

7.1 The data

- **`cases`:** 16 pairs (a, b), chosen to hit corner cases: all sign combinations, the most negative number, overflow, zero, divide by zero, large values.
- **`expected`:** 7 results per case (mul, mulh, mulhu, div, divu, rem, remu), 112 words. They were computed in Python with the RISC-V rules, then checked independently: when RARS runs the program, its own M instructions agree with every entry and the program's self-check gives `status = 0xFF`.

Case	a	b	Why
0–3	±7	±3	all sign combinations
4	0x80000000	−1	div overflow (result -2^{31}, remainder 0)
5	0x80000000	1	most negative number
6	0x7FFFFFFF	0x7FFFFFFF	largest positive product
7	−1	−1	signed 1 vs unsigned huge
8 / 9	0 / 5	5 / 0	zero dividend / zero divisor
10, 11	±123456789	1000	ordinary multi-digit
12	0x12345678	0x9ABCDEF0	large mixed
13	−2	2	divu of a "negative" number
14	1	0x80000000	divisor larger than dividend
15	0xDEADBEEF	0x00010001	pattern

7.2 Part 1: the self-check

For each case, the program runs all seven M instructions back to back, compares each with the table, and clears that instruction's LED bit on any mismatch:

```
beq  t0, a3, ok_mul
andi s3, s3, -2       # -2 = ...11111110: clears bit 0 (LED0)
ok_mul:
```

`andi` with −2, −3, −5, −9, ... clears bits 0, 1, 2, 3, ... respectively, because −2 is all ones except bit 0, −3 is all ones except bit 1, and so on.

After all 16 cases, LED7 is set if all seven flags survived, and the value is written to the LEDs.

7.3 Part 2: measuring cycles

```
lw  t0, CYCLECOUNT_OFF(s0)    # read the counter
lw  t1, CYCLECOUNT_OFF(s0)    # read again immediately
sub s4, t1, t0                # = 1: the cost of the measuring itself
lw  t0, CYCLECOUNT_OFF(s0)
mul t2, a0, a1
lw  t1, CYCLECOUNT_OFF(s0)
sub s5, t1, t0                # = 1 + 33 = 34
sub s5, s5, s4                # = 33: the mul alone
```

The cycle counter increments every CPU clock.

- Two back-to-back `lw` reads differ by 1 cycle, because the second `lw` executes 1 cycle after the first.
- With a `mul` in between, they differ by 1 + 33.
- Subtracting the "empty" measurement removes the measuring overhead, leaving exactly the M instruction's cycles.

This is why the board shows 0x21 = 33 for both: it's the hardware measuring itself. With the original signed multiplier (BOOTH = 0) the mul would measure 0x41 = 65, which is the Task 3 speed-up shown on the real FPGA.

7.4 Part 3: the display loop

- **SW15 = 0 (browse):**
  - Reads the case number from SW3–0, loads that case's a and b, and **recomputes** all seven results with M instructions (so the display is live hardware, not the table).
  - Stores them in a 16-word `view` array.
  - Shows `view[SW7–4]`. The array approach avoids a long chain of branches.
- **SW15 = 1 (own operands):**
  - `slli a0, t0, 17` then `srai a0, a0, 25` extracts SW14–8 as a signed 7-bit number. The left shift puts bit 14 at bit 31; the arithmetic right shift brings it back down while copying the sign: sign extension (Fact 2).
  - Then the buttons pick which operation to show.

7.5 What the board does

The bitstream turns the FPGA into the whole system: the RISC-V CPU with MCycle, instruction memory pre-loaded with `test.asm`, data memory pre-loaded with the tables, and the peripherals. The CPU clock is 100 MHz ÷ 2^5 = **3.125 MHz** (`CLK_DIV_BITS = 5` in TOP); the start-up self-check takes about 5,200 cycles (about 1.7 ms), so it looks instant.

- **After programming or BTND (reset):** LED7–0 all on (0xFF) if the self-check passed. LED0 mul, LED1 mulh, LED2 mulhu, LED3 div, LED4 divu, LED5 rem, LED6 remu, LED7 all.
- **LED15–9** show the PC (too fast to read); **LED8** is the CPU clock.
- **SW15 = 0:** SW3–0 = test case; SW7–4 = 0 a, 1 b, 2 mul, 3 mulh, 4 mulhu, 5 div, 6 divu, 7 rem, 8 remu, 9 mul cycles (`21` = 33), A div cycles (`21` = 33), B LED value.
- **SW15 = 1:** SW14–8 = a, SW6–0 = b (7-bit signed, −64…63). Shows a × b; hold BTNL for a ÷ b, BTNC for a rem b, BTNR for divu.
- **BTNU** pauses the CPU clock; **BTND** resets and reruns the self-check.

Suggested demo: point at the 8 LEDs; show case 4 view 5 ($-2^{31} \div -1$ = `80000000`); show view 9 (mul: 33 cycles, against 65 for the original signed multiplier); let the TA enter their own numbers with SW15 up.

---

8. How everything was tested

8.1 Self-checking testbenches

The original `test_MCycle.v` only applied a few inputs, and you had to read the waveforms to decide whether the answers were right. A **self-checking** testbench computes the expected answer itself and prints PASS or FAIL, so you can run thousands of tests without looking at a single waveform.

The expected answers come from Verilog's own `*`, `/` and `%` operators. Those are fine in a testbench because testbenches are never synthesised. It's an *independent* reference: a completely different implementation from the hardware being tested.

8.2 What it checks

- **Exhaustively at widths 4 and 6.** Every possible pair of operands, for all four operations. The algorithms are identical at every width (only the parameter changes), so proving them correct for *every* input at small widths is strong evidence for width 32.

- **At width 32:** a 10 × 10 grid of corner values (0, ±1, ±2, most negative, most positive, alternating bit patterns) plus 3,000 random pairs per operation.
- **Exact cycle counts.** Results arriving at the wrong time would break the CPU.
- **Busy rising in the same cycle as Start.** This is the property the stall depends on (6.4).
- **The ways the CPU can drive it:**
  - Start held high back to back (consecutive M instructions).
  - Start dropped for a cycle, checking the results are held while idle.
  - Operands changed randomly while Busy, checking the operand capture of 6.6.
- **A reset in the middle of an operation.**
- **Both versions,** BOOTH = 1 and 0, for the comparison.

Result: 76,083 checks pass.

Mutation testing. A testbench that passes proves little if it would also pass a broken design. So the design was deliberately broken in many different ways: wrong remainder sign, a Booth add on the wrong bit pair, a logical instead of arithmetic shift, M not sign-extended, the product taken from the wrong bits, one iteration short or long, operands not latched, and so on. The testbench failed every time, which shows it really can catch those bugs.

8.3 Post-synthesis simulation (Task 3 requirement)

- **Behavioural simulation** runs your Verilog as written.
- **Post-synthesis simulation** runs the netlist of LUTs and flip-flops that synthesis produced from it.

They can differ: a missing signal in a sensitivity list, an unintended latch, or reliance on initial values can make simulation and real hardware disagree. Running the same tests on the netlist proves the synthesised circuit really does what you simulated.

There's one complication. A synthesised netlist has no parameters left, because width and BOOTH were fixed during synthesis. So `test_MCycle_postsynth` instantiates MCycle *without* overriding parameters, using a Verilog `generate if` to choose between the two ways of instantiating. It was run on gate-level netlists of both versions; both pass, with signed multiply taking 33 and 65 cycles respectively.

Doing it in Vivado:

1. Set `MCycle` as top under Design Sources.
2. *Settings* → *Synthesis* → *More Options*: `-generic width=32 -generic BOOTH=1`.
3. Run Synthesis; note LUTs and FFs from *Report Utilization*.
4. Set `test_MCycle_postsynth` as the simulation top (`localparam BOOTH = 1`), run *Post-Synthesis Functional Simulation*, then `run all`.
5. Repeat with `BOOTH=0` and `localparam BOOTH = 0`.
6. Afterwards, remove the `-generic` options and set `TOP` / `test_Wrapper` back as tops.

8.4 CPU level

- **`test_Wrapper.v`** (229 checks) runs the real program on the whole CPU and checks:
  - the LEDs reach 0xFF,
  - every one of the 12 views for all 16 cases,
  - the measured cycle counts,
  - the own-operand mode with each button.

- `cpu_check.py` is the Assignment 2 test. It re-checks every RV32I instruction to prove that adding MCycle broke nothing; all 6,703 checks pass. Only MCycle.v was added to its file list and the M instructions to its model of the Decoder.

8.5 Vivado messages you can ignore

When MCycle is synthesised as top: the `TOP_Nexys4DDR.xdc` warnings (`No ports matched 'CLK_undiv'`, `No pins matched 'RX_MSF1_reg/C'`, `create_clock: No valid object(s)`) appear because the constraints are written for `TOP`'s ports; disable the XDC for a completely clean run, then re-enable it. The incremental-synthesis checkpoint warnings (`TOP.dcp is not suitable…`) are harmless; you can turn incremental synthesis off. `Too many words specified in data file AA_DMEM.mem` is normal (RARS dumps more data than DMEM holds).

---

9. Questions you're likely to be asked

Why does `mul` take 33 cycles, not 32? Booth reads one multiplier bit per cycle, so 32 iterations; then there's one extra cycle in which Busy falls and the result is written back. The original signed multiplier needed 64 iterations + 1 = 65.

Why is restoring division called "restoring"? When the trial subtraction R − D is negative, the subtraction is discarded and the old R is kept, i.e. restored.

Why does R need w+1 bits during the comparison? After the shift, R can be up to 2D − 1, which can need one more bit than D.

Why not just use `*` and `/`? `*` would map onto the FPGA's built-in DSP multipliers, which defeats the point of the assignment and is explicitly forbidden. `/` isn't synthesisable efficiently at all.

What happens if two M instructions are back to back? In the cycle where Busy falls, the state machine returns to IDLE regardless of Start, and the next instruction starts a fresh operation in the following cycle (6.7).

What does $-2^{31} \div -1$ give? -2^{31} with remainder 0. It falls out of the magnitude approach naturally (4.7).

Why gate the register write with ~Busy? So intermediate values are never written to rd, and the write happens exactly once, at the same edge where the PC advances.

How is A − M done in hardware? As A + (~M) + 1: invert M and set the adder's carry-in to 1 (two's complement negation, Fact 1). Synthesis builds `n_booth_A - n_booth_M` this way.

Why do A and M have w+1 bits? So that −M fits even when M is the most negative number (-2^{31}), and so A's sign stays correct after an add or subtract (5.5).

Why does the shift have to be arithmetic? A holds a signed partial result. Shifting right divides it by 2 and must keep its sign, so the sign bit is copied in (Section 1).

What does $\{Q_0, Q_{-1}\}$ = 11 or 00 mean? You are inside a run of 1s or a run of 0s, so nothing is added; only the shift happens.

What's the trade-off of your enhancement? Signed multiply is about 2× faster (65 → 33 cycles) and its adder is 33 bits instead of 64, but the Booth registers and adder are added alongside the template's, which unsigned multiply still uses, so the whole unit grows (about 12% more LUTs and 37% more flip-flops in the Yosys estimate). It is a speed-for-area trade-off. Processing 2 bits per cycle (radix-4 Booth) would roughly halve the cycles again, at the cost of a ±2M multiple and a select mux.

Practice: redo the two worked examples (13 ÷ 3 and −3 × 5) on paper, then try your own, such as −7 ÷ 2 or 5 × −2. Check multiplications step by step in the Booth visualisation, and both on the board in own-operand mode.